Predictive Maintenance for HVAC Pumps

This project implements a predictive maintenance system for HVAC pumps using sensor data to detect anomalies and predict potential failures. By leveraging machine learning, specifically a Random Forest Classifier, the system enables proactive maintenance, reducing downtime and operational costs in heating, ventilation, and air conditioning (HVAC) systems.

Project Overview

The objective is to analyze pump sensor data to identify patterns indicative of impending failures. The project includes data exploration, feature engineering, model development, and deployment of a prediction API. The deliverables include a comprehensive report, Python code, and a functional API endpoint for real-time predictions.

Key Features

- **Data Preprocessing**: Cleaning and preparing sensor data for analysis.
- **Feature Engineering**: Creating meaningful features to enhance model performance.
- **Random Forest Classifier**: A robust model for anomaly detection and failure prediction.
- **API Deployment**: A FastAPI-based endpoint for real-time predictions.
- **Actionable Recommendations**: Insights for maintenance teams to optimize pump operations.

Project Workflow

A[Load Pump Sensor Data] --> B[Data Exploration & Preprocessing] B --> C[Feature Engineering] C --> D[Train Random Forest Classifier] D --> E[Evaluate Model Performance] E --> F[Deploy Model via FastAPI] F --> G[Generate Recommendations]

Running the Model

1. Execute the Jupyter notebook (pump_predictive_maintenance.ipynb) to explore data, engineer features, and train the Random Forest Classifier.
2. Use the trained model to make predictions on new sensor data.

Accessing the API

- **Endpoint**: http://localhost:8000/predict
- **Method**: POST
- **Request Body** (example):

  Json

  {

  "sensor_1": 0.5,

  "sensor_2": 1.2,

  "sensor_3": 0.8,

  ...

  }

- **Response** (example):

  Json

  {

  "prediction": "Normal”

  }

Deliverables

1. **Data Exploration**: Visualizations (e.g., correlation heatmap) and summary statistics in the notebook.
2. **Feature Engineering**: List of engineered features (e.g., rolling averages, sensor ratios) with justifications.
3. **Model Development**: Comparison table of Random Forest vs. other models (e.g., Logistic Regression, SVM) and discussion of Random Forest's superior performance.
4. **Model Deployment**: FastAPI code (app.py) and API guide in API_Guide.md.
5. **Recommendations**: Actionable insights, such as maintenance schedules based on prediction thresholds.

Results

The Random Forest Classifier achieved:

- **Accuracy**: 95%
- **Precision**: 93% for failure predictions
- **Recall**: 90% for identifying anomalies

The model excels in handling imbalanced data and capturing complex sensor interactions, making it ideal for predictive maintenance.

Future Improvements

- Incorporate real-time streaming data for continuous monitoring.
- Experiment with deep learning models (e.g., LSTMs) for temporal patterns.
- Enhance API with authentication and rate limiting for production use.

Step-by-Step Implementation Plan

1. Data Understanding

We are using Pump Sensor Data

Goal: Predict machine_status (encoded), where:

- 0 = Normal
- 1 = Warning
- 2 = Failure

2. Data Preprocessing

- Cleaned the data (sensor_cleaned.csv)
- Encoded machine_status
- Split into X, y, then train_test_split()

3. Model Training

model = RandomForestClassifier(random_state=42)

model.fit(X_train, y_train)

4. Evaluation

- accuracy_score
- classification_report
- confusion_matrix + heatmap

5. Save Model and Features

import joblib
joblib.dump(model, "rf_model.pkl")
joblib.dump(list(X.columns), "feature_names.pkl")

6. Deploy with FastAPI (for Real-time Prediction)

Now we want a live API where sensor values can be passed and a prediction is returned (normal, warning, or failure).

Refer to this main.py file

7. Run the API

Use Terminal:
uvicorn main:app --reload

Go to: http://127.0.0.1:8000/docs

Test the /predict endpoint.